INTERNAL ASSIGNMENT

NAME	**SOHAIL ANSARI**
ROLL NO	**261450205242**
SESSION	**JULY 2026**
PROGRAM	**BACHELOR OF COMPUTER APPLICATIONS (BCA)**
SEMESTER	**I**
COURSE CODE & NAME	**DCA1108 Fundamentals of Computers & Digital Systems**

SET – 1

QUESTION -1: a) Explain the generations of computer system in detail.
b) What are the different types of memory used in computers to store data?

ANSWER – a) Explain the generations of computer system in detail

Computer generations are classified by the main technology used in their hardware:

First Generation (1940–1956) — Vacuum Tubes. Computers used vacuum tubes as switching devices, were very large, consumed enormous power, generated much heat, and used machine language. Memory was tiny and magnetic drums were used for storage. Input was through punched cards. Examples: ENIAC, EDVAC, UNIVAC-I.

Second Generation (1956–1963) — Transistors. The transistor replaced vacuum tubes, making computers smaller, faster, cheaper, and more reliable, with much lower power consumption. Assembly language and early high-level languages (COBOL, FORTRAN) appeared, along with magnetic core memory and magnetic tapes. Examples: IBM 1401, IBM 7094.

Third Generation (1964–1971) — Integrated Circuits (ICs). Many transistors were fabricated on a single silicon chip, which further reduced size and cost and increased speed. Keyboards, monitors, and operating systems were introduced, and multiprogramming became possible. Example: IBM System/360.

Fourth Generation (1971–1980s) — Microprocessors (VLSI). Large-scale integration placed an entire CPU on one chip. This made personal computers possible, along with GUIs, mice, and networks. Examples: Intel 4004, IBM PC, Apple Macintosh.

Fifth Generation (1980s–present) — Artificial Intelligence and Parallel Processing. Focus is on ULSI chips, AI, natural language processing, and parallel processing, with touch and voice interfaces, cloud computing, and machine learning.

Characteristics: each generation brought smaller size, lower cost, higher speed, greater reliability, and larger storage than the previous one.

b) What are the different types of memory used in computers to store data?

Computer memory is broadly divided into primary (main) memory and secondary (auxiliary) memory.

1. Primary Memory
   - RAM (Random Access Memory) — read/write, volatile; holds programs and data currently in use.
     - SRAM (Static RAM) — built from flip-flops, fast, no refresh needed, used for cache memory; costly.
     - DRAM (Dynamic RAM) — uses capacitors, needs periodic refreshing, slower and cheaper, used as main memory.
   - ROM (Read Only Memory) — non-volatile, contents retained when power is off.

    - PROM — programmed once by the user.
    - EPROM — erased by ultraviolet light and reprogrammed.
    - EEPROM — electrically erasable and reprogrammable.
    - Flash memory — a fast EEPROM variant used in USB drives, SSDs, and memory cards.
- Cache memory — small, very fast memory between CPU and RAM; stores frequently used data (L1, L2, L3).
- Registers — the fastest memory, located inside the CPU; hold the instruction, operands, and addresses during execution.

2. Secondary Memory

Non-volatile, large capacity, slower, used for permanent storage:

- Magnetic: hard disk drives (HDD), magnetic tapes.
- Optical: CD, DVD, Blu-ray.
- Solid-state: SSD, pen drives (no moving parts, faster, shock-resistant).

Memory hierarchy (fastest to slowest, costliest to cheapest): Registers → Cache → RAM → SSD/HDD → Optical/Tape. This hierarchy balances speed, cost, and capacity.

QUESTION -2: a) Explain various types of input and output devices used with computers. b) Compare and contrast the binary, decimal, octal, and hexadecimal number system.

ANSWER – a) Explain various types of input and output devices used with computers

Input Devices accept data and instructions from the user and convert them into a form the computer can process.

- Keyboard — most common; enters text, numbers, and commands.
- Mouse — pointing device for clicking, selecting, and dragging; variants include optical and wireless mice.
- Trackball and Touchpad — pointing devices used where space is limited (laptops).
- Joystick and Game Controller — for gaming and simulation.
- Light Pen and Stylus — for direct screen input and graphics tablets.
- Touch Screen — acts as both input and output; used in mobiles, ATMs, kiosks.
- Scanner — converts printed documents and photographs into digital images (flatbed, sheet-fed, handheld).
- OMR, OCR, MICR, Barcode Reader — specialised readers: OMR for objective exams, OCR for printed text, MICR for bank cheques, barcode readers at billing counters.
- Microphone — voice input.
- Digital Camera and Webcam — capture images and video.
- Biometric devices — fingerprint and iris scanners for authentication.
- Sensors — temperature, pressure, and motion sensors in embedded systems.

Output Devices present processed results to the user.

- Monitor (VDU) — soft-copy output; CRT (obsolete), LCD, LED, OLED types; measured by resolution and size.
- Printer — hard copy: impact (dot matrix, line printer) and non-impact (inkjet, laser, thermal).
- Plotter — large-scale drawings, maps, and engineering designs.
- Speaker and Headphones — audio output.
- Projector — displays output on a large screen.
- 3D Printer — produces physical objects from digital models.

Some devices, such as touch screens, modems, and network cards, perform both input and output functions. Input and output devices together make the computer interactive and useful.

b) Compare and contrast the binary, decimal, octal, and hexadecimal number system.

Basis	Binary	Decimal	Octal	Hexadecimal
Base (radix)	2	10	8	16
Digits used	0, 1	0–9	0–7	0–9, A–F
Number of symbols	2	10	8	16
Place value	Powers of 2	Powers of 10	Powers of 8	Powers of 16
Example	$(1011)_2 = 11$	11	$(13)_8 = 11$	$(B)_{16} = 11$
Use	Internal machine representation	Human calculations	Compact form of binary	Memory addresses, colour codes, MAC addresses

Explanation

The binary system uses only two digits, which maps directly onto the two physical states (ON/OFF, high/low voltage) of digital circuits — hence it is the native language of computers. The decimal system is the human system, but converting decimal fractions to binary can produce long, non-terminating representations, which makes it unsuitable for direct machine use.

The octal and hexadecimal systems are compact shorthand for binary, because $8 = 2^3$ and $16 = 2^4$. Hence one octal digit represents exactly three bits and one hexadecimal digit represents exactly four bits. For example:

- Binary 1011 1100 = Octal 274 = Hex BC = Decimal 188.

Conversions between octal/hexadecimal and binary are trivial (grouping of bits), whereas conversion between decimal and binary requires repeated division or multiplication.

Contrast summary: binary is simple but long; decimal is familiar but not machine-friendly; octal is compact but less used now; hexadecimal is the most practical compact notation, widely used in programming, debugging, and digital design.

QUESTION -3: a) Explain NAND and NOR as universal gates with their symbols and truth tables.

b) Minimize the given Boolean Expression by using the four-variable K-Map.

F (A, B, C, D) = Σ m (2, 4, 5, 6, 12, 13, 14) + d (1, 3).

ANSWER – a) Explain NAND and NOR as universal gates with their symbols and truth tables.

A universal gate is a gate using which any other logic gate (AND, OR, NOT) can be constructed. NAND and NOR are the two universal gates.

1. NAND Gate

The NAND gate is an AND gate followed by a NOT gate; its output is the complement of AND. Symbol: an AND gate with a small circle (bubble) at the output. Boolean expression: $Y = (A \cdot B)'$

A	B	A·B	Y = (A·B)′
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

NAND as universal gate:

- NOT: tie both inputs together → Y = (A·A)′ = A′
- AND: NAND followed by NOT (using NAND as inverter)
- OR: invert both inputs with NAND, then feed to NAND → Y = A + B

2. NOR Gate

The NOR gate is an OR gate followed by a NOT gate. Symbol: an OR gate with a bubble at the output. Boolean expression: Y = (A + B)′

A	B	A+B	Y = (A+B)′
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

NOR as universal gate:

- NOT: tie both inputs → Y = (A + A)′ = A′
- OR: NOR followed by NOT
- AND: invert both inputs with NOR, then feed to NOR → Y = A·B

Significance: Since NAND and NOR can implement all Boolean functions, complete digital systems can be built using only one type of gate. This reduces the number of IC types needed, simplifies manufacturing, and lowers cost — which is why NAND and NOR are the most widely fabricated gates in CMOS and TTL logic families.

b) Minimize the given Boolean Expression by using the four-variable K-Map.

F(A,B,C,D) = Σm(2,4,5,6,12,13,14) + d(1,3). (5)

Step 1 — Plot the K-Map (rows AB = 00, 01, 11, 10; columns CD = 00, 01, 11, 10 in Gray-code order):

AB \ CD	00	01	11	10
00	0 (m0)	X (m1)	X (m3)	1 (m2)
01	1 (m4)	1 (m5)	0 (m7)	1 (m6)
11	1 (m12)	1 (m13)	0 (m15)	1 (m14)

AB \ CD	00	01	11	10
10	0 (m8)	0 (m9)	0 (m11)	0 (m10)

Here X denotes don't-care conditions (m1, m3), which may be treated as 0 or 1 to form larger groups.

Step 2 — Form groups

- Group 1 (Quad): m4, m5, m12, m13 → A varies, B = 1, C = 0, D varies ⇒ BC′
- Group 2 (Quad): m4, m6, m12, m14 → A varies, B = 1, C varies, D = 0 ⇒ BD′
- Group 3 (Pair): m2, m6 → A = 0, B varies, C = 1, D = 0 ⇒ A′CD′

Step 3 — Verify coverage

Minterm	Covered by
m2	A′CD′
m4	BC′, BD′
m5	BC′
m6	BD′, A′CD′
m12	BC′, BD′
m13	BC′
m14	BD′

All seven minterms are covered, and no group includes a cell containing 0. The don't-care cells were not required, so they remain unused.

Step 4 — Minimized expression

F(A,B,C,D) = BC′ + BD′ + A′CD′

Alternative valid form: F = BC′ + BD′ + A′B′C (grouping m2 with don't-care m3 gives A′B′C instead of A′CD′). Both give 7 literals; the first is preferred because it reuses the existing 1 at m6.

Observations: grouping as quads (rather than pairs) removes two variables per group, producing the minimum number of literals; don't-care conditions increase grouping flexibility but are used only if they help form larger groups.

SET – 2

QUESTION -4: A) DIFFERENTIATE BETWEEN COMBINATIONAL AND SEQUENTIAL CIRCUIT. B) IMPLEMENT HALF SUBTRACTOR AND FULL SUBTRACTOR USING BASIC LOGIC GATES. DRAW THEIR LOGIC DIAGRAMS AND TRUTH TABLES

ANSWER - A) DIFFERENTIATE BETWEEN COMBINATIONAL AND SEQUENTIAL CIRCUIT

Basis	Combinational Circuit	Sequential Circuit
Definition	Output depends only on the present inputs.	Output depends on present inputs outputs (state).
Memory element	None	Requires memory (flip-flops)
Clock	Not required	Usually clocked
Feedback	No feedback path	Feedback path exists
State	No internal state	Has internal state
Design complexity	Simpler	More complex
Speed	Faster (no clock delay)	Slower (clock and propagation del
Ease of testing	Easier	Harder
Examples	Adders, subtractors, multiplexers, decoders, encoders, comparators	Flip-flops, registers, counters, fin machines, memory

Explanation

In a combinational circuit, the output is a pure Boolean function of the inputs — the same inputs always produce the same output. For example, a half adder produces Sum = $A \oplus B$ and Carry = $A \cdot B$, irrespective of any previous operation.

In a sequential circuit, the output depends on the current input sequence as well as the current state stored in memory elements. The clock synchronises state changes. For example, a counter increments only on each clock pulse, so its output depends on how many pulses have occurred — that is, on history.

Practical significance: combinational circuits form the data-processing (arithmetic and logic) part of a digital system, while sequential circuits provide control, storage, and timing. Real systems, such as a CPU, combine both — an ALU (combinational) with registers and control logic (sequential).

B) IMPLEMENT HALF SUBTRACTOR AND FULL SUBTRACTOR USING BASIC LOGIC GATES. DRAW THEIR LOGIC DIAGRAMS AND TRUTH TABLES

1. Half Subtractor

A half subtractor subtracts one bit B from another bit A and produces a Difference (D) and a Borrow (Bout). It has two inputs and two outputs and does not account for a borrow from a previous stage.

Truth Table

A	B	D = A − B	Bout
0	0	0	0
0	1	1	1

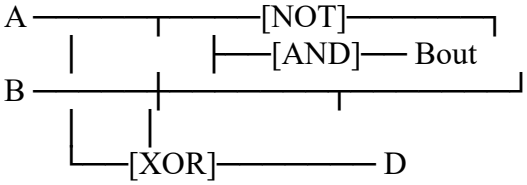
A	B	D = A − B	Bout
1	0	1	0
1	1	0	0

Boolean Expressions

- $D = A'B + AB' = A \oplus B$
- $Bout = A'B$

Logic Diagram (gate description):

text



It uses one XOR gate (for D) and one NOT gate plus one AND gate (for Bout).

2. Full Subtractor

A full subtractor subtracts B and an incoming borrow Bin from A, producing Difference and Borrow out. It has three inputs and two outputs and can be cascaded to subtract multi-bit numbers.

Truth Table

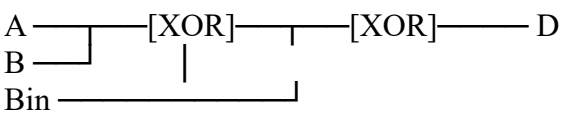
A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Boolean Expressions

- $D = A \oplus B \oplus Bin$
- $Bout = A'B + A'Bin + B \cdot Bin$ (equivalently $Bout = A'B + Bin(A' + B)$)

Logic Diagram (gate description):

text



Bout = A′B + A′Bin + B·Bin → three 2-input AND gates
+ one 3-input OR gate
(A′ obtained from one NOT gate)

A full subtractor can also be built from two half subtractors plus an OR gate: the borrow outputs of the two half subtractors are OR-ed to give the final borrow.

Application: Half and full subtractors are the basic building blocks of binary subtraction and arithmetic logic units; cascade full subtractors to build an n-bit parallel subtractor, and combine adders with subtractors (using 2's complement) for combined arithmetic hardware.

QUESTION -5: a) What are flip-flops? Explain SR and JK flip-flops with their truth tables. b) Explain synchronous counter. Describe its working principle and purpose. Differentiate it from an asynchronous counter.

ANSWER – a) What are flip-flops? Explain SR and JK flip-flops with their truth tables

Flip-Flops

A flip-flop is a bistable sequential logic circuit that stores one bit of data. It has two stable states (0 and 1) and retains its state until an input causes it to change, making it the fundamental memory element in registers, counters, and RAM. Flip-flops are edge-triggered (change on the rising or falling edge of a clock) and are built from cross-coupled NAND or NOR gates. Common types: SR, JK, D, and T flip-flops.

1. SR (Set–Reset) Flip-Flop

It has two inputs — S (Set) and R (Reset) — and outputs Q and Q′.

S	R	Q(next)	Meaning
0	0	Q	No change (hold)
0	1	0	Reset
1	0	1	Set
1	1	Invalid	Forbidden state

Characteristic equation: Q(next) = S + R′Q, with the constraint S·R = 0.

The S = R = 1 condition is a race condition/forbidden state — both outputs try to become 1, and when inputs return to 0, the final state is unpredictable.

2. JK Flip-Flop

The JK flip-flop removes the forbidden state of the SR flip-flop by feeding back the outputs, so that toggling occurs instead.

J	K	Q(next)	Operation
0	0	Q	Hold
0	1	0	Reset
1	0	1	Set
1	1	Q′	Toggle

Characteristic equation: Q(next) = JQ′ + K′Q

When J = K = 1, the output toggles on every clock pulse — this is the basis of counters and frequency dividers. The Master–Slave JK flip-flop eliminates the racing problem by using two stages triggered on opposite clock levels, ensuring only one state change per clock cycle.
Comparison: SR is simpler but has a forbidden state; JK is more versatile, is free of the forbidden state, and supports toggle operation, which makes it the preferred choice in counter and register design.

b) Explain synchronous counter. Describe its working principle and purpose. Differentiate it from an asynchronous counter.

Synchronous Counter

A synchronous counter is a sequential circuit in which all flip-flops are clocked simultaneously by the same common clock signal. Because every flip-flop receives the clock edge at the same instant, all outputs change together on that edge.

Working Principle

In a synchronous binary counter, the first flip-flop (LSB) toggles on every clock pulse. The next flip-flop toggles only when all preceding lower-order outputs are 1. This condition is implemented with AND/steering gates feeding the J and K (or T) inputs of each flip-flop, so that a flip-flop toggles only when required. For example, in a 3-bit synchronous up counter:

- FF0 (LSB): J = K = 1 → toggles every clock
- FF1: J = K = Q0
- FF2: J = K = Q0·Q1

The combinational logic is designed so that each stage predicts whether it must toggle, rather than waiting for a ripple from earlier stages. Any desired sequence (up, down, or arbitrary) can be produced by designing the next-state logic.

Purpose

Synchronous counters are used for frequency division, event counting, digital clocks and timers, address generation, and as program counters in digital systems. They are preferred in high-speed designs because the output changes are simultaneous and glitch-free.

Synchronous vs Asynchronous Counter

Basis	Synchronous	Asynchronous (Ripple)
Clock	Common clock to all flip-flops	Only first FF clocked; output of each FF clocks the next
Output change	All outputs change simultaneously	Outputs change one after another (ripple)
Propagation delay	Only one FF delay — small	Cumulative delay; grows with number of bits
Maximum speed	Higher	Lower for long counters
Circuit complexity	Higher (extra AND/steering logic)	Simpler, fewer gates
Cost	Higher	Lower
Decoding glitches	Minimal	Present during ripple — false counts possible
Best use	High-speed and precise counting	Simple, low-cost, low-speed applications

QUESTION-6: WHAT ARE SHIFT REGISTERS? EXPLAIN TYPES OF SHIFT REGISTERS.

ANSWER - Definition

A shift register is a sequential logic circuit consisting of a group of flip-flops connected in cascade, in which binary data is shifted from one flip-flop to the next on each clock pulse. A register with *n* flip-flops can store *n* bits. Shift registers are used for data storage, data transfer (serial ↔ parallel conversion), arithmetic operations (multiplication and division by powers of 2), and sequence generation. They may be unidirectional (left or right shift) or bidirectional.

Basic Working

Each flip-flop has its output connected to the input of the next flip-flop, and all (or the first) receive clock pulses. On each clock edge, the contents shift by one position, and one new bit enters at the input end while one bit is lost at the output end.

Types of Shift Registers

1. Serial-In Serial-Out (SISO) Register

Data is entered one bit at a time at the serial input and exits one bit at a time at the serial output. A 4-bit SISO register requires 4 clock pulses to load data and 4 more to output it. It acts as a digital delay line and is used in serial communication to delay data by *n* clock cycles.

text

Input → [FF] → [FF] → [FF] → [FF] → Output

2. Serial-In Parallel-Out (SIPO) Register

Data enters serially, but after *n* clock pulses all *n* bits are available simultaneously at parallel outputs. It performs serial-to-parallel conversion and is used at the receiving end of serial communication links, in demultiplexing, and to drive parallel displays.

3. Parallel-In Serial-Out (PISO) Register

All bits are loaded simultaneously through parallel inputs, then shifted out one bit at a time. It performs parallel-to-serial conversion, commonly used at the transmitting end of serial links and in communication interfaces such as UART.

4. Parallel-In Parallel-Out (PIPO) Register

Data is loaded in parallel and read out in parallel; no shifting occurs. It simply functions as a buffer or temporary storage register (for example, an accumulator or data latch).

5. Bidirectional Shift Register

Using additional control logic, data can be shifted either left or right, selected by a mode control signal. Such registers are used in arithmetic operations — left shift multiplies by 2 and right shift divides by 2.

6. Universal Shift Register

A versatile register that supports all four operations — SISO, SIPO, PISO, and PIPO — plus left and right shifting, selected by control inputs. It is the most flexible form and is widely used in general-purpose digital systems.

7. Rotate (Circular) Shift Register

The output of the last flip-flop is fed back to the input of the first, so bits circulate without being lost. Used in ring counters and sequence generation.

Applications

- Serial data communication: SIPO at the receiver, PISO at the transmitter.
- Data conversion: serial-to-parallel and parallel-to-serial.
- Arithmetic: multiplication and division by powers of 2, and serial addition.
- Delay lines and temporary storage.
- Sequence generators: ring and Johnson counters.
- Keyboard and display scanning in embedded systems.

Conclusion

Shift registers are simple yet highly versatile sequential circuits. Their ability to store, delay, convert, and manipulate data in serial and parallel forms makes them essential building blocks in communication systems, arithmetic units, and general-purpose digital hardware.